

Vibration Monitoring — Where We Stand

A plain-language account of what has been built, what it found, and what it needs

10 September 2026
No technical background assumed

In one paragraph: We built software that reads vibration measurements from a pump and writes a report about the machine's condition. **The software works.** What is holding it back is the quality of the measurements being fed into it — not the software itself. Two of the data problems we found have been fixed. Three others need somebody to go to the plant and take a proper measurement; no amount of programming can substitute for that.

1. Why this project exists

Machines that spin — pumps, motors, fans — vibrate. A healthy machine vibrates in a steady, predictable way. As parts wear out, the vibration *changes*, and it changes **weeks or months before the machine actually fails**.

If you can spot that change early, you repair the machine on a planned shutdown for a modest cost. If you miss it, the machine fails during production, and you pay for the emergency repair plus the lost output.

So: sensors on the machine record the vibration, and this project is the software that reads those recordings and says what they mean.

2. What we built

Three separate pieces of software. We call them "agents" — each does one job and can be run on its own.

THE AGENT	WHAT IT DOES	THINK OF IT AS
The librarian	Answers technical questions using 8 professional reference books that have been loaded in (13,059 sections indexed). Ask it a question and it finds the answer and tells you which book and which page it came from.	A very well-read colleague who always cites their source
The data fetcher	Pulls the sensor measurements out of the main system and hands them over in a usable form.	The person who goes to the archive and brings back the files
The report writer	Takes those measurements, works out what they mean, and writes the condition report.	The engineer who reads the data and writes the assessment

3. The one rule everything was built around

This is the most important design decision in the project, and it is worth understanding even if nothing else is.

Modern AI writes fluent, confident, professional-sounding text. It also, occasionally, makes numbers up. Not wildly — it produces a number that looks entirely reasonable, in the right place, in the right units. Which is exactly what makes it dangerous: nobody reading the report can tell.

We tested this during the project. When we simply *instructed* the AI to be careful, it got things right most of the time — but not always. In one test it reproduced a formula correctly only 2 times out of 5.

So we took a different approach: the AI is not allowed to write any number at all.

Every figure is recorded first — where it came from, how it was worked out, what it means — and the report can only *refer* to figures on that list. If any number appears in the report that is not on the list, **the software refuses to produce the report.** Not a warning in the corner. It stops.

After that change, the same formula test passed 5 times out of 5.

The lesson generalises, and it is the main thing we learned: **asking the AI to behave gives you "usually". Enforcing it in code gives you "always"**. Anything that has to be right cannot be left to instructions.

4. What happened when we ran it on a real machine

We ran it against Cooling Water Pump 1, using every measurement the system had recorded.

WHAT WAS CHECKED	RESULT
Recordings available	9
Measuring points on the machine	8
Individual health checks performed	720
Figures in the final report, every one traceable to its source	36
Automated tests protecting all of this	223

The report was produced successfully and passed its own self-check — every number in it traces back to a recorded measurement.

But the report cannot yet tell you whether the pump is healthy. Not because the software failed, but because of what it found in the data. That is the honest position, and the report says so on its own front page rather than producing a confident-looking assessment it cannot support.

5. What we fixed

FIXED The machine's speed had gone missing

To assess a spinning machine you must know how fast it spins. Every other calculation depends on it. The field meant to hold that number was empty on all 9 recordings, so it looked as though the speed was simply unknown and most of the analysis was impossible.

It turned out the speed was never missing. The main system had already worked it out from the vibration and stored it. But the piece of software that carried the data across was built with a fixed list of what to carry — and the speed was not on that list, so it was quietly dropped in transit.

We fixed the list. The speed came back: **1500 rpm**

And here is the good part. The manufacturer's plate on that pump says **1480 rpm**. Our calculation never looks at that plate — it works purely from the vibration, and it landed on 1500. Two completely independent routes to essentially the same answer.

Stated precisely: this method reports speed in steps of about 187 rpm, so 1480 and 1500 are the same step and it could not have printed 1480 even if that were exact. What the agreement proves is that we are on the correct step — not double it, not a third of it, not noise. That is the check that matters, and it passed. It is not a claim of 1.4% accuracy.

FIXED No way to convert into the international standard's units

There is an international standard that says how much vibration is acceptable for a machine of a given type. To use it, our readings must be in the same units the standard uses. They were not, and there was no conversion.

We built the conversion. But it deliberately **refuses to guess** one thing, and we consider that refusal a feature rather than a limitation.

The stored measurements are just numbers — nothing anywhere records what those numbers physically represent. There are two possibilities, and they differ by a factor of ten. Rather than pick one and produce a confident answer that might be ten times wrong, it asks, and if not told, it leaves that section out and says why.

6. What is still blocking us

Three problems remain. **None of them can be solved by writing more software.** Each needs a real measurement taken at the plant.

BLOCKED There is no "before" photograph

Think of checking a wall for fading. You photograph it when freshly painted, photograph it again months later, and compare. The first photo is your reference.

We have no first photo of this pump. There is a file marked as the reference, **but it is the same recording as the one being tested** — we are comparing today's photo against today's photo. The answer comes back "no change" every time, not because the machine is fine but because no genuine "before" was ever taken.

What is needed: somebody records the pump on a day it is running normally, and marks that recording as the reference. Only a person who knows the machine was healthy at that moment can do this. The software cannot work it out.

This blocks 144 of the 720 checks. The remaining 576 do not need a reference and work correctly today.

BLOCKED One set of alarm limits for every machine on the site

The system currently allows only a single alarm limit per measurement type, shared by every machine. There is nowhere to record "this particular pump is different".

The consequence is easy to see. The alarm limit for one measurement is set to 0.02. The readings from this pump run from 0.024 to 0.424 — **the very lowest reading is already over the limit, and the highest is 21 times over.** So every single reading is flagged critical.

That does not mean the pump is 21 times too rough. It means **the limit was never set for this pump.** A limit that no reading has ever satisfied is describing itself, not the machine.

What is needed: a change to how the system stores limits, so each machine can have its own. This is development work on the main platform, not on our agents.

BLOCKED The nine recordings are one recording, loaded nine times

We compared all 9 recordings against each other. Every one contains **numerically identical values.** It is the same test file, uploaded repeatedly.

This means there is no history. You cannot see a trend, a rate of change, or predict anything from a single snapshot copied nine times.

The report shows trend figures as *"cannot be assessed"* rather than as a flat line — because a flat line would look like a beautifully stable machine, when really it is the same file nine times.

What is needed: genuine recordings taken on different days.

7. How the three problems relate

The third one is the root. Getting real recordings from the pump over a period of time would: give us a history to trend, provide a candidate for the "before" photograph, and settle an open question about the measurement units.

One activity — taking real measurements — unblocks two of the three problems. Only the alarm-limits issue is genuinely separate, and that one is platform development.

8. Where things stand

ITEM	STATUS	WHO CAN MOVE IT
The three agents (librarian, data fetcher, report writer)	WORKING	Done

ITEM	STATUS	WHO CAN MOVE IT
Protection against invented numbers	WORKING	Done
Machine speed recovered and independently confirmed	FIXED	Done
Conversion to the standard's units	FIXED	Done
A healthy "before" recording	BLOCKED	Plant — someone records the pump when it is running well
Per-machine alarm limits	BLOCKED	Platform development team
Real recordings over time	BLOCKED	Plant — normal measurement routine
Connecting the report to existing machine records	IN PROGRESS	Us — small piece of work, next up
Two documents (this one and the technical one)	IN PROGRESS	Us

9. What we would ask for

1. **A recording of the pump while it is running normally**, with someone confirming the machine was in good condition at the time. This is the single highest-value thing anyone can give this project.
2. **Recordings taken regularly** — weekly or monthly — rather than the same file loaded repeatedly. This is what makes prediction possible at all.
3. **A decision on per-machine alarm limits**, since that is platform work rather than something we can do inside the agents.

The honest summary: the analysis software is built, tested and working, and it refuses to state anything it cannot prove. It is currently reporting mostly on the quality of its input data, which is the correct behaviour for the data it has been given. Give it real measurements and it will report on the machine.